

Separating Disclosure from Authorization:

Field-Tier Minimization for Agent Action Mediation

Jiten Oswal John Cadeddu

Aurite AI

jiten@aurite.ai john@aurite.ai

ORCID (J. Oswal): 0009-0009-8866-869X

August 25, 2026

Abstract

A system that authorizes an action must see enough of it to decide, and a system that attests to its decision must record enough to be audited. Both pressures push raw action parameters — recipients, payment memos, record identifiers — into an append-only ledger that cannot delete them. We show the two are separable. We classify each parameter *field*, not each action class, into three tiers: fields a policy may legitimately match on, which cross raw; fields that are policy-relevant but identifying, which cross only as projections such as an email domain or a templated route shape; and fields with no legitimate policy use, which never leave the workload. The central property is that the ledger’s commitment is a canonical digest of the *full, unminimized* parameters, computed *before* minimization runs. The commitment is therefore independent of the tier table: reclassifying a field changes what is disclosed without invalidating a historical entry, reopening a hash, or altering what an offline verifier checks. Tier table, policy schema and wire schema are generated from one per-action declaration, so the deciding and recording parties cannot hold different rules. We then address a question the architecture forces: *which party should compute each attested fact?* We argue it is settled by which party could lie about it undetectably, and derive three answers within one request — the client computes the parameter digest, being the only party holding the data; it is structurally prevented from naming the definition that governed it, since that would write a false statement into a signed ledger; and it attests which tier table it applied, so divergence is detectable. We give a leakage analysis of each projection, report an incident in which a first-cut projection preserved the identifier it was written to remove, and state the residual trust the design does not eliminate.

1 Introduction

Consider an autonomous agent that sends an email on a customer’s behalf. To authorize it, a policy engine must evaluate a rule — *may this agent email outside the company?* — which requires knowing something about the recipient. To audit it, the system must record what happened, durably, in a form an auditor can check months later. Both requirements are legitimate. Both, implemented naively, place the recipient address, the subject line, and the body in a mediation service and then in an append-only log.

That log is the problem. An audit ledger built for tamper-evidence is deliberately hard to change: rows are hash-linked and signed, deletion breaks the chain, and retention is measured in years [12, 27]. Whatever enters it is there for the life of the record. A system that governs agents therefore concentrates, in its most immutable component, precisely the data its customers are most regulated about [16], and does so as a *side effect* of doing its job well.

The standard responses are unsatisfying. Redacting at write time means the ledger commits to something other than what happened, and an auditor cannot tell what was removed. Hashing

everything means no policy can match on anything. Encrypting the payload moves the problem to key management and leaves the plaintext recoverable by whoever holds the key, which for an audit ledger is the party whose behaviour is under audit.

Our claim. The two pressures are separable, and the separation is cheap if the commitment is placed correctly. The policy engine needs *predicates over* parameters, not parameters. The ledger needs a *commitment to* parameters, not parameters. Neither needs the values themselves to cross the boundary, and the values that must cross for policy reasons need not be the identifying ones.

This paper describes a design that exploits this, implemented and running in a runtime governance system for enterprise AI agents. That system’s overall architecture — and the argument for why governing agents is a runtime problem rather than a build-time or alignment one — is described separately [8]; minimization is one mechanism inside its governance layer, and this paper is self-contained with respect to it. The system is operated as a small number of private pilots rather than as a general release. That is a deliberate choice we return to in section 9.3, because it determines what this paper can and cannot claim.

Contributions.

1. **Field-level, not class-level, minimization** (section 3), with a three-tier vocabulary and a declassifying projection library.
2. **Digest independence** (section 4): the ledger commits to a digest of the raw parameters computed before minimization, which makes the disclosure policy *retrofitable and re-tierable* over an immutable ledger.
3. **One declaration, three artifacts** (section 5): the tier table, the policy schema, and the wire schema are generated from one per-action declaration, so the deciding party and the recording party cannot disagree.
4. **An asymmetry principle for attested facts** (section 6): *the party that computes an attested fact should be the party that cannot lie about it undetectably*. This yields three different answers within a single request, and getting one wrong produces a false statement in a signed ledger, a failure strictly worse than a wrong authorization decision.
5. **Disclosure is not authorization** (section 7): deny-by-default protects one and not the other.
6. **A leakage analysis** (section 8) of each projection: what each still reveals, a quantification over our corpus, and an incident in which a first-cut projection preserved the identifier it was written to remove.

Section A traces one request end to end (raw parameters, digest, projected attributes, policy evaluation, attested event) using values pinned as literals in our test suite. A reader who wants the concrete artifact before the argument should start there.

We do not claim novelty for tiered data classification, nor for canonical hashing. The contribution is the *placement of the commitment*, the asymmetry principle it forces, and the demonstration that together they make disclosure policy mutable over an immutable record.

2 Setting

2.1 The mediation path

An agent attempting a governed action calls an authorization endpoint before the action takes effect. The mediator evaluates a policy (in our implementation, Cedar [10]) against an entity model in which the agent is the principal, the action type is the action, and the action’s parameters

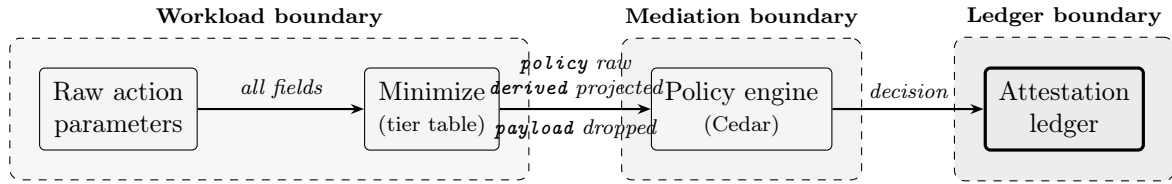


Figure 1: The three boundaries. Exposure at the mediation boundary is bounded in time and audience; exposure at the ledger boundary is neither, because the record is permanent and designed to be exported to third parties. Treating them as one boundary is the error this design avoids.

supply *resource attributes*. The decision is returned to the agent, which proceeds or does not, and the decision is written to an append-only, hash-linked, signed ledger.

The parameters are therefore load-bearing twice: once as policy inputs, once as the audited record of what was attempted.

2.2 Trust boundaries

Three boundaries matter, and conflating them is the mistake this design exists to avoid (fig. 1).

The workload boundary. The agent runs in the customer’s environment. Code we ship runs inside it, as an SDK. Data that never leaves the process has not crossed any boundary at all. That is the strongest position available, and the one the *payload* tier occupies.

The mediation boundary. The policy engine may run in the customer’s account or ours. Either way it is a different failure domain from the workload, and anything crossing it is disclosed in the sense that matters: it exists somewhere the workload does not control.

The ledger boundary. Anything written here is effectively permanent and is designed to be exportable to third parties, which is the point of an evidence package. Data crossing this boundary is disclosed to every future auditor.

A design that treats “the mediator is trusted” as equivalent to “the ledger may hold anything” has collapsed the second and third boundaries, which are not the same boundary.

2.3 What “minimization” must not break

Three properties were non-negotiable, and they constrain the design more than the privacy goal does.

- **The attested record must commit to what actually happened**, not to a redacted view of it. An auditor must be able to establish that a specific parameter set produced a specific decision.
- **Existing chain entries must remain verifiable**, byte-for-byte, under an offline verifier that has already been published for independent use.
- **The policy language must not become weaker in ways operators cannot see.** If a policy can no longer express something, that must be visible at authoring time, not as a silent non-match at runtime.

3 Field-tier minimization

3.1 The unit is the field

The natural first design classifies *action types*: `send_email` is sensitive, `read_data` is not. This fails immediately on inspection of any real action. `send_email` carries a recipient (identifying, and policy-relevant), a subject (identifying, and in our corpus referenced by zero policies), an attachment count (not identifying, and policy-relevant), and a body (never policy-relevant under any rule we have seen). A per-class decision must either disclose the subject to permit a recipient rule, or refuse recipient rules to protect the subject. Both are wrong.

So the unit of classification is the field, classified once, statically, per action type.

3.2 Three tiers

Tier	Rule	Crosses as
<code>policy</code>	A policy may legitimately match on it, and it does not identify a person or record	The raw value
<code>derived</code>	Policy-relevant, but the raw value identifies	A non-identifying projection
<code>payload</code>	No legitimate policy use	Nothing — folded only into the digest

The `derived` tier is the interesting one and the reason field-level classification is worth its cost. It exists because the *policy-relevant* part of an identifying value is usually a strict, computable abstraction of it. “*May this agent email outside the company?*” is a question about a domain; the local part identifies a person and answers nothing.

In information-flow terms [14, 29] the tiers are a two-point lattice with an explicit declassification set: `payload` is high, `policy` is low, and each `derived` field carries a named declassifier mapping high to low. The design is a *declassification policy* [30], and much of what follows, particularly section 6.3, is about making that policy itself auditable.

3.3 The projections

Five projections cover our corpus. Each is pure, individually unit-tested, and compiled into every client.

`recipient_domain` The substring after the last `@`, lowercased. A recipient with no `@` has no domain and yields `""`, which matches no domain policy. Note the direction of that failure: an unparseable input fails to a value that *denies* under allowlist policies rather than to the raw value.

`payee_known` A boolean: is this payee in the operator’s allowlist? The payee itself never crosses. An absent allowlist is treated as empty, so every payee is unknown, which again fails closed.

`endpoint_host` The lowercased host of an external endpoint. The implementation prefers the WHATWG URL parser but only trusts it when it yields a real authority, because `new URL("host:port/p")` parses `host` as a *scheme* with an empty hostname; scheme-less service identifiers fall back to an authority/path split.

`endpoint_path_template` The route *shape*: the path with concrete instance identifiers replaced by `{id}` and the query string dropped. Numeric segments, UUIDs and long hex tokens are treated as identifiers; version-like segments (`v1`) and named routes (`patients`, `records`) are preserved, because they are the route’s meaning rather than its subject. Thus

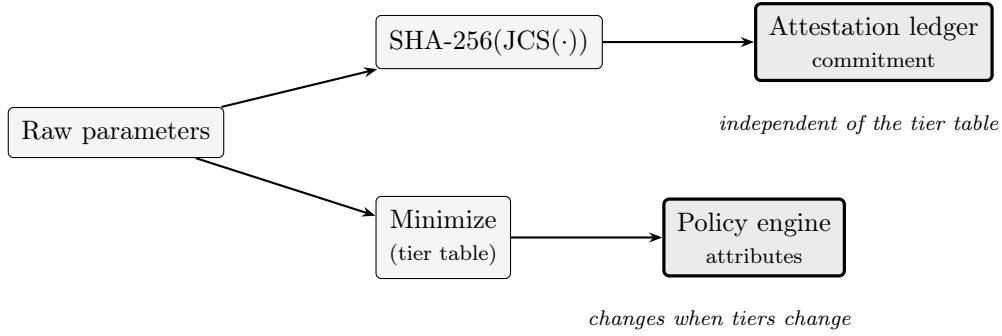


Figure 2: The fork. Both computations take the raw parameters and share nothing else. Because the commitment is taken *before* minimization, reclassifying a field moves the lower path only — no historical hash is reopened and the offline verifier is untouched.

`https://api.x.com/v1/patients/40b1.../records?token=abc`
 $\rightarrow$ `/v1/patients/{id}/records`

resource_path_prefix The *directory* a resource lives in, with identifier-like segments templated and the final segment removed entirely. This is the projection that got it wrong the first time, and section 8.4 is about that.

3.4 Two safety rules that make incompleteness harmless

Unclassified means payload. A field a definition does not mention is dropped, not passed. There is deliberately no “default tier” parameter anywhere in the API. This is what makes a half-finished action definition leak nothing: the failure mode of forgetting to classify a field is that a policy about it silently does not match — visible to the operator, fixable — rather than that the field is disclosed, which is neither.

An unknown projection is refused, not approximated. A definition naming a projection the client build does not have is rejected *for that field*. Applying the closest available projection instead would substitute one disclosure decision for another without anyone deciding to.

Both rules follow the same principle: when the system is unsure what a field is, the safe answer is the one that reduces disclosure, even at the cost of function. This is the mirror image of deny-by-default, applied to a dimension deny-by-default does not cover (section 7).

4 Digest independence

4.1 The property

The attested event records `action_parameters_hash` and `action_parameters_size_bytes`. Both are computed by the client, over the full original parameters, *before* minimization runs:

$$\text{action_parameters_hash} = \text{SHA-256}(\text{JCS}(\text{parameters}))$$

where JCS is RFC 8785 canonical JSON serialization [28] and SHA-256 is as specified in FIPS 180-4 [15]. Minimization then runs separately, producing the Cedar attributes that cross the boundary. The two computations share an input and nothing else (fig. 2).

The consequence is the central property of this design:

The ledger’s commitment does not depend on the tier table.

Reclassifying a field — moving `subject` from `payload` to `policy`, or narrowing `resource_path` from a full path to a directory — changes what crosses the boundary and changes nothing about what the ledger committed to.

4.2 Why this matters more than it first appears

An immutable ledger and a mutable disclosure policy are, on their face, incompatible. If the commitment were computed over the *minimized* view, then every tier change would fork the meaning of the hash: entries before and after would commit to differently-shaped objects, an offline verifier would need the tier table as of each entry’s date to interpret it, and the tier table would itself become an input to verification, and therefore something an attacker could attack.

Placing the commitment before minimization avoids all of that:

- **Tier changes are purely additive.** No historical entry’s hash changes.
- **The offline verifier is untouched.** It already shipped; it verifies signatures and chain linkage over bytes, and those bytes do not move.
- **There is no version-dependent hash to get wrong.** The design is *attach a pointer*, never *redefine a commitment*.
- **Minimization is retrofittable.** We added it to a system with a populated chain, and entries written before it remain valid, comparable and verifiable against entries written after.

The last point is the practical one. A design that required minimization from day zero would be a design nobody could adopt.

4.3 What the commitment does and does not prove

The ledger entry proves: *a client holding parameters whose canonical digest is H requested action A , and the mediator decided D .* An auditor in possession of a candidate parameter set can confirm it is the one: the digest is over the raw values, so the check is exact and requires no knowledge of the tier table.

It does not prove that the mediator *saw* those parameters; by construction it did not. Section 9.2 is about the residual trust that follows.

5 One declaration, three artifacts

5.1 The drift problem

The tier table is not the only artifact describing an action. Three exist: the policy schema (the Cedar entity types and attribute names a policy may reference), the tier table, and the wire schema (runtime validation of an incoming request).

Written independently, they drift, and each drift is a distinct failure. A field in the policy schema but not the tier table is a policy that can never match. A field in the tier table but not the wire schema is an attribute accepted without validation. Worst, a field classified `policy` in the server’s copy and `payload` in a client’s copy means the two parties disagree about what may be disclosed, and the disagreement is invisible, because each behaves correctly by its own table.

That last case is not hypothetical. Our Python SDK carried a duplicated tier table, and it drifted.

5.2 Generation from one declaration

All three artifacts are now generated from a single per-action declaration: each field’s tier, its schema, its Cedar type, and, for **derived** fields, the projections it maps to. The server and the primary client SDK import that declaration rather than restating it.

Our second-language SDK is the honest qualification. It implements the tier vocabulary and the digest, and cross-language conformance is enforced by tests comparing digests produced by both implementations over a shared corpus, so digest divergence fails in CI. It does *not* yet implement the runtime definition distribution or the applied-tiers attestation of section 6.3. The result is that our strongest guarantee — that the deciding and recording parties cannot hold different disclosure rules — currently holds for one client language and is enforced by conformance testing rather than by construction for the other. We state this because a generation argument that quietly excludes a shipped client is the kind of claim this paper is otherwise arguing against.

5.3 An aside on testing generators, which we got wrong first

The generator’s first safety argument was an *equivalence* test: assert the generated tier table matches the hand-written one. That test was correct, and it became worthless the moment the hand-written table was deleted — at which point it compared the generator to itself. Forty-nine assertions, all vacuously true, sitting in CI looking like coverage. The failure is the operational analogue of vacuous satisfaction in model checking [5, 21], where a property passes because its antecedent never occurs.

The replacement is a golden corpus of literals: every expected value written out by hand, captured from the reviewed implementation before the swap. Literals have independent force precisely because a human wrote them down, and they keep that force when the generator is itself replaced. The relationship to mutation testing [13, 19] is close: what makes a check meaningful is evidence that it can fail.

The leak guard in that corpus is worth describing, because the obvious version of it does not work. It does not check that identifying values are absent from the keys they belong to; it serializes the entire minimized output and asserts the identifying values appear nowhere in it, under any key. The failure being guarded is a projection that returns its input, which a per-key assertion would miss entirely, since the value would appear exactly where a correct implementation puts a correct value.

6 Who computes what: an asymmetry principle

Minimization forces a question that a non-minimizing mediator never has to ask. If the server does not see the raw parameters, it cannot compute facts about them, so some attested facts must be computed by the client. Which ones?

The party that computes an attested fact should be the party that cannot lie about it undetectably. Where that is impossible, the fact must not be attestable by the client at all.

Applied to a single authorization request, this yields three different answers.

6.1 The parameter digest: the client computes it, and is trusted

Only the client holds the raw parameters. No other party can compute the digest. The server therefore records what the client reports.

This is a real trust concession and we name it as such: a client that lies about its own digest produces a ledger entry that commits to parameters other than the ones it used. Four things bound it.

The claim is not anonymous. The digest arrives over a mutually-authenticated channel on which the client presents a short-lived credential issued to its workload by an attestation-based identity plane [31], on the strength of platform properties it cannot choose for itself. The ledger therefore does not record *some client asserted H*; it records *this attested workload asserted H*.

The party who could forge it is the party it would harm. The client is the customer’s own workload in the customer’s environment, and the ledger exists to evidence that workload’s behaviour to third parties. Forging a digest is forging evidence about oneself.

And it is independently checkable by exactly that party. A customer who wishes to verify the claim can log raw parameters locally and recompute: the digest is over the raw values, requires no knowledge of the tier table, and the computation is public. The residual is invisible to *us* and visible to *them*, which is the correct direction for a control the customer relies on.

The threat model is unaffected. What this system defends against is an operator — including us — altering the record after it is written. The chain’s tamper-evidence properties [12] concern exactly that and are untouched by who computed the digest.

The alternative is worth stating: for the server to compute the digest, the server must receive the raw parameters. The residual is the price of the property this paper is about, and there is no version of the design that has both. Section 9.2 describes the principled way to buy it back.

6.2 The action definition: the client is structurally prevented

An attested payload also records the *definition* that governed the action: which fields existed, which crossed, what a policy could have referenced. This is what makes an old entry interpretable.

A client that could supply this field could attest that its action was governed by a definition that never applied. That is worse than a wrong decision. It is a false statement written into a signed, append-only ledger, and a wrong decision at least shows up as a wrong decision, where this does not show up at all. The forged definition would be indistinguishable from the real one, permanently, and would corrupt exactly the interpretive layer an auditor depends on.

So the definition is derived by the server from its own registry, and every client-facing request schema is *strict*: an unknown key is rejected, not ignored. A client cannot supply the field because it cannot supply any field the schema does not name.

Two details make this a property rather than an accident. First, strictness was adopted for unrelated reasons, so nothing stated that this field in particular must never be accepted — a test now states it. Second, and more important, the test’s load-bearing half is its coverage check rather than the five schemas it checks: the list of client-facing request bodies is derived from the module’s exports and compared against what is tested, so a sixth request body added later fails the test until it is covered. The failure mode of a hand-listed guard is the case nobody added to the list.

6.3 The applied tier table: the client attests, the server compares

Between “client is trusted” and “client is prevented” lies the interesting case.

The client applies a tier table to produce the minimized attributes. The server knows which table it *should* have applied. Neither can verify the other directly: the server cannot see the raw parameters, and the client cannot see the registry.

But both can compute a digest over the *rules*. The client sends SHA-256(JCS(*applied-tiers view*)), a canonical reduction of the definition it actually applied: each field’s tier, its normalized optionality, and its sorted projection names. The server computes the same digest over the definition it authorized against and compares.

A mismatch means the two minimized under different rules: a stale client, a registration that landed mid-flight, or a definition tampered with in transit. The design decision we consider correct is that this is a fact about disclosure, so it belongs in the chain rather than in a log line.

Two implementation details generalize.

The view is normalized, not hashed as given. `optional: undefined` and `optional: false` must not produce different digests; projection names are sorted. Otherwise the two sides disagree for reasons that have nothing to do with tiers.

The view is rebuilt rather than hashed as received. The SDK holds the wire shape, which may carry keys the contract does not cover: a description field today, something else tomorrow. Hashing those would make the two sides disagree about serialization rather than about disclosure. A digest intended to detect one kind of divergence must be computed over exactly that kind and nothing else, otherwise its false-positive rate destroys its value, and a control that cries wolf gets turned off.

6.4 The principle, generalized

Fact	Computed by	Why
Parameter digest	Client, trusted	Only party holding the data
Action definition	Server only	A client value would be undetectably false
Applied tier table	Client, <i>compared</i>	Neither verifies the other's inputs; both share the rules

The middle row is the one designs get wrong, because the field is *convenient* for the client to supply and the client already supplies its neighbours. The test is not convenience or even trust in the client; it is what a false value would look like afterward. **A fact whose falsification would be undetectable in the permanent record must not be client-supplied at any level of trust.**

7 Disclosure is not authorization

7.1 Deny-by-default protects one and not the other

Our mediator is deny-by-default: an action no policy permits is refused. This is a complete protection for the authorization question. It is *no* protection at all for the disclosure question, and the asymmetry is easy to miss because the same table drives both.

Re-tier a field from `payload` to `policy` and the raw value begins crossing the boundary on every subsequent request, with no policy changed, nothing redeployed, and no denial anywhere to signal it. In the policy corpus we classified against, the highest-value sensitive field is `send_email.subject` — free text, and referenced by none of the policies in that corpus. A single re-tier of that one field starts protected content flowing into the ledger on every send. Deny-by-default has nothing to say about this, because nothing was denied. (The corpus is our own reference policy set, not a customer's; we make no claim about the distribution in the wild.)

Field tiering is a disclosure decision made with the same table as the authorization decisions, and it needs controls of its own.

7.2 The trust inversion

The problem sharpens when the catalog becomes tenant-extensible. Before, the tier table was code the customer shipped and reviewed, so a change required a release. After, it is data the

server sends, refreshed at runtime. An administrator re-tiering a field in a console changes what every agent discloses, immediately, with no code review anywhere in the loop.

This is a genuine regression in the customer’s control, created by a feature that is otherwise clearly right. Naming it is the first obligation, and the rest of this section answers it.

7.3 The narrow-only workload cap

The answer is a *disclosure floor*: an optional, workload-side declaration that caps what this workload will ever send, whatever the catalog says. The catalog may narrow disclosure further; it may never widen past the floor.

The floor can only push a field *down* to payload. There is deliberately no way to raise one. A configuration that could widen disclosure would be a second place to make exactly the mistake the floor exists to prevent, and two mechanisms that can both widen are worse than one.

It is off by default , and we think this is right despite being the weaker security posture. A floor that had to be configured before anything worked would make the extensible catalog useless, and a mechanism nobody enables protects nobody. The trade is named in the design record rather than hidden.

The general shape — *the party bearing the risk holds a veto that can only reduce exposure* — is what we would reach for again.

8 Leakage analysis

A projection is only as good as what it fails to remove. This section states, per projection, what an observer of the minimized attributes learns. We use “identifying” in the ordinary re-identification sense [33], and note that the standard caution about auxiliary information [23] applies to every row below.

8.1 Per-projection

recipient_domain. Discloses the recipient’s organization. For a large public provider this is near-zero information; for a small or uniquely identifying domain it can approach identification of the counterparty, though not of the individual. *Residual:* the domain is a strong feature in aggregate, since a sequence of decisions reveals an organization’s correspondents over time.

payee_known. One bit. Discloses whether a payee is on the operator’s allowlist and nothing about who they are. This is the strongest projection in the set and the model for others: the policy question was always a membership test, so the projection is exactly the predicate.

endpoint_host. Discloses which third-party service was called. Frequently this *is* the sensitive fact. Calling a specialist medical API discloses category information about the subject even with every identifier removed. We have no mitigation and do not claim one; a policy about which services an agent may call cannot be enforced without knowing the service.

endpoint_path_template. Discloses route structure, not instances. *Residual:* a route whose *shape* is unique to a purpose leaks that purpose, and segments that are semantically identifying but syntactically ordinary (a username in a path segment, say) are preserved, because the templating rule is syntactic. The honest statement is that the rule catches identifier *formats*, not identifier *meanings*.

resource_path_prefix. Discloses the directory, not the leaf. *Residual:* a directory hierarchy that encodes subject identity leaks category, in that `/data/patients/{id}/` still tells you the record is a patient record.

8.2 The corpus, quantified

Our reference corpus classifies 19 fields across 5 action types:

Tier	Fields	Share
policy (crosses raw)	11	58%
derived (crosses projected)	6	32%
payload (never crosses)	2	11%

The two **payload** fields are an email subject and a payment memo: both free text, both referenced by no policy in the corpus.

The headline number is not the 11%. Read carelessly, two-of-nineteen suggests minimization barely does anything. The number that matters is that 8 of 19 fields, 42%, never cross in their raw form, because the six **derived** fields are precisely the identifying ones: the recipient address, the payee, the resource path, the external endpoint, the attachment list. A per-class design would have had to choose between disclosing all six and refusing every policy that depends on them.

The six **derived** fields produce seven projected attributes, because one field (the endpoint) projects to two independent policy-relevant abstractions: its host and its route shape. That asymmetry is the argument for making the projection a named function per target attribute rather than a per-field transform.

What this quantification is and is not: it describes the corpus we classified against — our own reference policy set (section 7.1) — and is offered so the shape of the classification is checkable, not as evidence about the distribution in other organizations’ policies. Section 9.3 states what we can and cannot generalize.

8.3 What the tier table itself discloses

The applied-tiers digest (section 6.3) is a hash over the *rules*, not the data. It discloses nothing about parameters. But the tier table itself, once tenant-extensible, is a description of what an organization considers sensitive — mild but non-zero information, and it lives in our registry.

8.4 An incident: the projection that returned its input

Our first **resource_path_prefix** templated identifier-like segments across the whole path and kept the leaf. It was tested and it looked right. It failed on this repository’s own sample resource identifier: `patient-record-778812`.

Segment-wise templating left it completely intact, because the segment is not wholly numeric and not a UUID: the identifier is embedded in a string with other characters. The URL projection gets away with per-segment templating only because URLs conventionally give an identifier its own segment; filesystem resource identifiers do not follow that convention.

The leak guard described in section 5.3 caught it, specifically because it searched the serialized output for the identifying value rather than checking the field it expected the value in. The fix was to drop the final segment entirely rather than attempt to sanitize it, on the reasoning that the leaf is where the identifier lives, and a projection that tries to clean a string is a projection whose correctness depends on enumerating formats.

What that costs, stated plainly: a policy can no longer discriminate on a filename or extension. A rule reaching only `*.csv` files under `/data/patients/` cannot be written; the rule can reach `/data/patients/{id}/` and no further. We accepted it because filenames are precisely

where record identifiers and personal names appear (a file named for its subject defeats every containment rule written about the directory holding it), and because the containment questions customers actually ask us are questions about directories: *may this agent read outside the records it was scoped to?* The extension was never the control; the location was.

We report this because it is the general lesson of the exercise: a projection that attempts to remove sensitive substrings is fragile in a way that a projection which discards a whole structural component is not. Prefer dropping a component to cleaning one.

9 Limits and residual trust

9.1 Limits of the mechanism

The mediator’s decision rests on projections, so it can be wrong in ways the raw path would not be. A policy that would have matched on a full path may not match on a directory. This is visible at authoring time — the attribute simply does not exist in the schema — which is the design’s answer, but an answer that depends on operators reading schemas.

Nothing here defends against a compromised workload. An agent whose process is controlled by an attacker discloses whatever it likes over its own network connections; minimization governs what crosses *our* boundary, not every boundary.

The tier table is a static, per-field, per-action classification. It cannot express context-dependence (*this field is sensitive when the recipient is external*), and we have not needed it. We expect that need to arrive, and the design has no answer for it today.

9.2 Residual trust, and the principled way to remove it

The ledger commits to the client’s claim about its own parameters (section 6.1). An auditor can confirm a candidate parameter set matches the digest; they cannot establish that the client did not compute the digest over something else. The mediator cannot check this, by construction. What the design does provide is that the *disclosure rules* under which the client operated are attested and compared (section 6.3), and that the *definition* is beyond the client’s reach (section 6.2).

We think the principled way to close this is known and we have not built it. What is wanted is a proof, verifiable by the mediator, that the minimized attributes are a correct application of a stated tier table to *some* preimage of the committed digest, a zero-knowledge proof of correct projection [17, 25]. That would make the digest and the attributes jointly checkable without the mediator learning the parameters, which is precisely the property section 4 obtains structurally and cannot obtain cryptographically. The obstacles are practical rather than theoretical: the projections include string operations that are expensive to express as arithmetic circuits, and the mediation path is latency-sensitive. We regard this as the natural next piece of work on this design and the place where the structural approach of this paper meets the cryptographic one.

9.3 Deployment scope, and what it means for these claims

The system is deployed as a small number of private pilots, not as a general release. This is a deliberate choice rather than a stage we are passing through: what a governance layer must express is determined by what organizations actually need governed, and that is learned by working closely with a few deployments rather than broadly across many. A control plane that reaches general availability before its designers understand the requirement tends to have made the requirement up.

What we do claim. The design is implemented, running, and exercised by tests that pin every tier assignment against hand-written literals and compare digests across two language implementations. The failures reported in section 5.1 and section 8.4 are real and were found by these mechanisms. The properties argued in section 4 and section 6 are structural — they follow from where the commitment is computed and which party can supply which field — and do not depend on deployment scale.

What we do not claim. We have not shown that the three tiers are sufficient for policy corpora other than the one we classified against, nor that our five projections cover the space of policy-relevant abstractions, nor anything about operator behaviour at scale, in particular whether the disclosure controls of section 7 are ones administrators actually reach for. The floor of section 7.3 is off by default, and we have no evidence about how often it is enabled, because there is not yet a population to measure. These are questions about adoption, and they need a larger and more varied deployment base than we have.

9.4 How the design arrived at its present shape

Because a design presented as a finished argument can look as though it was reasoned out in advance, we record what actually happened. The system has been under development since May 2026, and the minimization design in this paper has been through four generations. Each was forced by a specific finding rather than by a change of taste.

	Change	What forced it
1	A hand-written per-field tier table, a Cedar schema derived from it, and client-side digest builders in two languages	The per-class alternative had no good setting (section 3.1)
2	Resource paths project to a <i>directory</i> rather than a templated full path	The leak in section 8.4: an identifier survived segment-wise templating intact
3	Tier table, policy schema and wire schema <i>generated</i> from one declaration; the equivalence test guarding the generator replaced by golden literals	Three hand-maintained artifacts had already drifted once (section 5.1), and the generator’s own safety argument had gone vacuous (section 5.3)
4	The client <i>attests the tier table it applied</i> ; the server compares (section 6.3)	Asking whether the client and the server could disagree about a definition, and finding that they could not, because the client had none

Two of these four were reversals of something we had already reviewed and shipped, and one — the fourth — is the paper’s strongest contribution and was not in the original design at all. It arrived from a question rather than from a plan.

We report this because the asymmetry principle of section 6 in particular reads as though it were derived top-down. It was not. It was extracted from three decisions taken separately, for local reasons, over several weeks, and the principle became visible only once they sat beside each other.

10 Related work

Information-flow control and declassification. The tier vocabulary is a lattice [14] and the **derived** projections are declassifiers, which places this work in the tradition of language-based information-flow security [29, 22]. Our setting is narrower and the mechanism correspondingly cruder: classification is per-field and static rather than derived from program analysis, and

enforcement is a library boundary rather than a type system. What we add to the picture is the *attestation* of which declassification policy was applied (section 6.3), which is not usually a concern in that literature [30] because the policy is compiled in.

Data minimization in regulation and practice. The minimization principle is a legal requirement [16] without an implementation, and the engineering literature on privacy by design [18, 26] supplies goals more readily than mechanisms. Most engineering responses operate at the storage layer: redaction, tokenization, field-level encryption. The distinguishing feature here is that minimization occurs *before* the trust boundary rather than at rest, so the data does not need protecting because it does not arrive.

Accountability logging and redaction. Designing logs to evidence compliance while limiting what they retain is a recognised problem [6, 7], and structured-logging practice typically classifies at the log-statement level, the class-level design we rejected in section 3.1. Redaction at write time additionally breaks the commitment property of section 4: an entry then commits to a redacted view, and a reader cannot tell what was removed.

The cryptographic form of this idea is closer to our concerns and deserves an explicit comparison. *Redactable* and *sanitizable* signature schemes [32, 20, 4] allow a signed document to have parts removed while the remainder still verifies, which sounds like exactly what an evidence package wants. The difference is where the sensitive value goes. Those schemes let a holder redact a record *after* signing, so the data must first exist in signed form: it has crossed the mediation and ledger boundaries of section 2.2 and been committed to by a signer who saw it. Our concern is upstream of that: the identifying value never reaches the signer at all, and what is committed is a digest computed where the data already lives. The two are complementary rather than competing — redactable signatures govern what a holder may disclose from a record, and this design governs what enters the record — and a system wanting both would apply them at different points.

Transparency logs and certificate transparency supply the append-only, externally verifiable ledger model [27, 12]. To our knowledge they do not address the tension between an immutable record and a mutable disclosure policy, because the logged objects are public by construction.

Agent governance, and why this is a runtime problem. Work on governing autonomous agents at runtime is early but no longer absent: a recent survey maps the agent-identity standards landscape and its gaps [24], and composition and provenance for agentic systems are receiving dedicated treatment [3]. We are not aware of prior work on the disclosure question this paper addresses. The nearest adjacent work is build-time: static validation can enforce agent patterns and catch classes of defect before code ships [1]. It cannot constrain what a probabilistic agent chooses to do with parameters it composes at runtime, which is the case addressed here: an agent’s action is selected by a model rather than programmed, so the parameters that must be authorized and audited do not exist until the moment of the call. That runtime framing, and the decomposition of the problem into which this mechanism fits, are set out in [8]. The agent framework this governance layer was built alongside [2] is representative of that class. We note the distinction rather than claim the ground: build-time validation and runtime mediation are complements, and a deployment wanting assurance needs both.

Privacy-preserving authorization — attribute-based and anonymous credentials [11, 9], and zero-knowledge proofs of policy satisfaction [17] — solves a stronger version of this problem with cryptographic guarantees rather than structural ones. The comparison is unflattering to us in rigour and flattering in deployability: our projections are ordinary functions any auditor can

read, run and reason about without a trusted setup, and the whole system is a library change rather than a protocol change. We regard section 8’s honest residuals as the price of that.

11 Conclusion

The mediation point of an agent governance system is where the pressure to see everything and the pressure to record everything meet, and it is also the point where those pressures are most easily mistaken for one requirement. They are not. A policy engine needs predicates; a ledger needs a commitment; neither needs values, and the values that must cross for policy reasons are usually not the identifying ones.

Separating them turns out to hinge on one placement decision. Compute the commitment over the raw parameters, before minimization, and disclosure policy becomes mutable over an immutable record: tiers can be reclassified, minimization can be retrofitted to a populated chain, and an already-shipped offline verifier never learns that any of it happened. Compute it over the minimized view and none of that is available.

The second lesson was less expected. Minimization redistributes *who knows what*, and therefore forces a question about who may attest to what. The answer we arrived at was derived from a specific near-miss: the right party is the one who cannot lie undetectably, and a fact failing that test must be structurally unavailable to the client rather than merely discouraged. We suspect it generalizes to any system that both minimizes and attests.

The last lesson is the one we would tell a team starting this work: **the tier table is a disclosure decision, and deny-by-default does not protect it**. Every control we built for the authorization half was already there and worked. Every control for the disclosure half had to be built separately, and the fact that one table drives both is exactly why it took us an incident to notice.

Data availability

A reference implementation is in preparation and will be released under the MIT licence: the five projections of section 3.3, the per-action tier declaration format of section 5.2, and the applied-tiers digest of section 6.3. The purpose of releasing it is that section 8’s leakage claims should be checkable rather than believed: each projection is a pure function of a few lines, and a reader who disagrees with our analysis can run it. It is not available at the time of writing.

It is intended as a frozen artifact accompanying this paper, not a supported library. We expect the projection set to grow in our own system and make no undertaking to keep the published version in step.

The mediator’s internals, the policy corpus, and the catalog registry API are deliberately not included. They are not part of the verification surface: nothing in section 8 depends on them, and a reader cannot check any claim in this paper more thoroughly by having them.

A A request, end to end

Every value below is taken from the golden corpus described in section 5.3, which pins them as literals in the test suite. Nothing here is illustrative.

A.1 send_email

The agent calls the SDK with:

```
{
  "action_type": "send_email",
  "recipient": "Jane.Doe@Customer.COM",
```

```

"subject": "patient 778812 discharge summary",
"body_size_bytes": 2048,
"attachment_hashes": ["aaaa...(64 hex)", "bbbb...(64 hex)"]
}

```

Step 1, commit over the raw parameters. The SDK computes $\text{SHA-256}(\text{JCS}(\text{parameters}))$ over the object exactly as given, before anything else happens. This is the value the ledger will carry, and section 4’s whole argument is that it is fixed at this moment, independent of every classification decision below.

Step 2, minimize. The tier table for `send_email` classifies four fields: `recipient` (derived), `body_size_bytes` (policy), `attachment_hashes` (derived), `subject` (payload). What crosses the workload boundary is:

```

{
  "recipient_domain": "customer.com",
  "body_size_bytes": 2048,
  "attachment_count": 2
}

```

Three things happened. The recipient was projected to its domain, lowercased, so `Jane.Doe` did not cross. The attachment *hashes* became a *count*, since no policy in the corpus asks which files, only how many. And the subject — “*patient 778812 discharge summary*”, the highest-value sensitive value in the whole request — did not cross at all, and does not appear in the ledger except as an input to the Step 1 digest.

The corresponding test asserts that the strings `Jane.Doe`, `patient 778812 discharge summary` and `778812` appear nowhere in the serialized output, under any key (section 5.3).

Step 3, attest the disclosure rules. The SDK computes the applied-tiers digest over the tier table it used and sends it alongside (section 6.3). The server computes the same digest over the definition it authorized against and compares.

Step 4, evaluate. Cedar sees only the three attributes above. A policy `recipient_domain == "customer.com"` matches; a policy about the subject line cannot be written, because that attribute does not exist in the generated schema, which is the design’s answer to silent non-matching (section 2.3).

Step 5, attest. The event records the action type, the decision, the matched policy, the applied-tiers digest, and the Step 1 hash. An auditor holding a candidate parameter set can confirm it is the one, exactly, including the subject line that never left the workload.

A.2 The projections on harder input

Raw	Projected	Note
/data/patients/ 40b1c2d3-.../rec.json	/data/patients/{id}/	UUID templated, leaf dropped
patient-record-778812	<i>attribute omitted</i>	No directory at all
nobody-at-example (no @)	""	Unparseable → matches no domain policy

The second row is worth dwelling on. A resource with no directory emits no attribute, rather than an empty one. Absent beats empty: Cedar’s `has` then means what it says, and a policy

written like "*" cannot quietly match a resource whose directory was never known. It is also the input that defeated the first implementation (section 8.4).

References

- [1] J. Oswal. *Agent Verifier: catch security issues, enforce standards, and validate agent patterns before code ships*. Open-source software, 2026. <https://github.com/Aurite-ai/agent-verifier>
- [2] J. Oswal, B. Randle, and R. Wilcox. *Aurite AI Agents Framework: a Python framework for building and orchestrating AI agents*. Open-source software, 2025. <https://github.com/Aurite-ai/aurite-agents>
- [3] S. Dutta and A. K. Moharir. AgentRiskBOM: a risk-scoping security bill of materials for agentic AI systems, 2026. arXiv:2606.21877.
- [4] G. Ateniese, D. H. Chou, B. de Medeiros, and G. Tsudik. Sanitizable signatures. In *ESORICS*, 2005. doi:10.1007/11555827_10.
- [5] I. Beer, S. Ben-David, C. Eisner, and Y. Rodeh. Efficient detection of vacuity in temporal model checking. *Formal Methods in System Design*, 18(2):141–163, 2001. doi:10.1023/A:1008779610539.
- [6] D. Butin, M. Chicote, and D. Le Métayer. Log design for accountability. In *IEEE Security and Privacy Workshops (International Workshop on Data Usage Management, DUMA)*, pages 1–7, 2013. doi:10.1109/SPW.2013.26.
- [7] D. Butin and D. Le Métayer. Log analysis for data protection accountability. In *19th International Symposium on Formal Methods (FM)*, volume 8442 of *LNCS*, pages 163–178. Springer, 2014. doi:10.1007/978-3-319-06410-9_12.
- [8] J. Oswal and J. Cadeddu. Five primitives for governing autonomous AI agents at runtime, 2026. Companion paper.
- [9] J. Camenisch and A. Lysyanskaya. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In *EUROCRYPT*, 2001. doi:10.1007/3-540-44987-6_7.
- [10] J. W. Cutler, C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, E. Ioannidis, J. Kastner, A. Mamat, D. McAdams, M. McCutchen, N. Rungta, E. Torlak, and A. M. Wells. Cedar: A new language for expressive, fast, safe, and analyzable authorization. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1):670–697, 2024. doi:10.1145/3649835. Extended version: arXiv:2403.04651.
- [11] D. Chaum. Security without identification: transaction systems to make big brother obsolete. *Communications of the ACM*, 28(10):1030–1044, 1985. doi:10.1145/4372.4373.
- [12] S. A. Crosby and D. S. Wallach. Efficient data structures for tamper-evident logging. In *USENIX Security Symposium*, pages 317–334, 2009. https://www.usenix.org/legacy/events/sec09/tech/full_papers/crosby.pdf
- [13] R. A. DeMillo, R. J. Lipton, and F. G. Sayward. Hints on test data selection: Help for the practicing programmer. *IEEE Computer*, 11(4):34–41, 1978. doi:10.1109/C-M.1978.218136.
- [14] D. E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976. doi:10.1145/360051.360056.
- [15] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. FIPS PUB 180-4, 2015. doi:10.6028/NIST.FIPS.180-4.
- [16] European Parliament and Council. Regulation (EU) 2016/679 (General Data Protection Regulation), Article 5(1)(c): data minimisation, 2016. <https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- [17] S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989. doi:10.1137/0218012.

- [18] S. Gürses, C. Troncoso, and C. Diaz. Engineering privacy by design. In *Conference on Computers, Privacy and Data Protection (CPDP)*, 2011. <https://software.imdea.org/~carmela.troncoso/papers/Gurses-CPDP11.pdf>
- [19] Y. Jia and M. Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi:10.1109/TSE.2010.62.
- [20] R. Johnson, D. Molnar, D. Song, and D. Wagner. Homomorphic signature schemes. In *CT-RSA*, 2002. doi:10.1007/3-540-45760-7_17.
- [21] O. Kupferman and M. Y. Vardi. Vacuity detection in temporal model checking. *International Journal on Software Tools for Technology Transfer*, 4(2):224–233, 2003. doi:10.1007/s100090100062.
- [22] A. C. Myers and B. Liskov. A decentralized model for information flow control. In *ACM Symposium on Operating Systems Principles (SOSP)*, 1997. doi:10.1145/268998.266669.
- [23] A. Narayanan and V. Shmatikov. Robust de-anonymization of large sparse datasets. In *IEEE Symposium on Security and Privacy*, 2008. doi:10.1109/SP.2008.33.
- [24] T. Otsuka, K. Toyoda, and A. Leung. AI identity: standards, gaps, and research directions for AI agents, 2026. arXiv:2604.23280.
- [25] B. Parno, J. Howell, C. Gentry, and M. Raykova. Pinocchio: Nearly practical verifiable computation. In *IEEE Symposium on Security and Privacy*, 2013. doi:10.1109/SP.2013.47.
- [26] A. Pfitzmann and M. Hansen. A terminology for talking about privacy by data minimization: anonymity, unlinkability, undetectability, unobservability, pseudonymity, and identity management. Technical report, v0.34, TU Dresden and ULD Kiel, 2010. https://dud.inf.tu-dresden.de/literatur/Anon_Terminology_v0.34.pdf
- [27] B. Laurie, A. Langley, and E. Kasper. Certificate Transparency. RFC 6962, IETF, 2013. doi:10.17487/RFC6962.
- [28] A. Rundgren, B. Jordan, and S. Erdtman. JSON Canonicalization Scheme (JCS). RFC 8785, IETF, 2020. doi:10.17487/RFC8785.
- [29] A. Sabelfeld and A. C. Myers. Language-based information-flow security. *IEEE Journal on Selected Areas in Communications*, 21(1):5–19, 2003. doi:10.1109/JSAC.2002.806121.
- [30] A. Sabelfeld and D. Sands. Declassification: Dimensions and principles. *Journal of Computer Security*, 17(5):517–548, 2009. doi:10.3233/JCS-2009-0352.
- [31] SPIFFE Project. *Secure Production Identity Framework for Everyone (SPIFFE): specification*. <https://spiffe.io>.
- [32] R. Steinfeld, L. Bull, and Y. Zheng. Content extraction signatures. In *ICISC*, 2001. doi:10.1007/3-540-45861-1_22.
- [33] L. Sweeney. *k*-anonymity: A model for protecting privacy. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 10(5):557–570, 2002. doi:10.1142/S0218488502001648.